Name: Devarjya
Reg No: 24968040

lab 7 Q2) Write a program in CUDA which performs convolution operation on the input N elements using a mask array with M elements to produce the resultant N elements

Code:

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void convolution_1d(const float *input, const float *mask, float *output, int N, int M) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N) {
        float sum = 0.0f;
        int radius = M / 2;

        for (int j = 0; j < M; j++) {
            int input_idx = i - radius + j;

            if (input_idx >= 0 && input_idx < N) {
                sum += input[input_idx] * mask[j];
            }
        }

        output[i] = sum;
    }
}

int main() {
    printf("Name: Devarjya, Reg No: 240968040\n");
    int N = 10;
    int M = 3;

    float h_input[] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f, 9.0f, 10.0f};
    float h_mask[]  = {0.25f, 0.5f, 0.25f};
    float h_output[10];

    float *d_input, *d_mask, *d_output;

    cudaMalloc((void**)&d_input, N * sizeof(float));
    cudaMalloc((void**)&d_mask, M * sizeof(float));
    cudaMalloc((void**)&d_output, N * sizeof(float));

    cudaMemcpy(d_input, h_input, N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_mask, h_mask, M * sizeof(float), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
```

```
    printf("Launching CUDA kernel...\n");
    convolution_1d<<<blocksPerGrid, threadsPerBlock>>>(d_input, d_mask, d_output, N, M);

    cudaMemcpy(h_output, d_output, N * sizeof(float), cudaMemcpyDeviceToHost);

    printf("\nResult of 1D Convolution:\n");
    for (int i = 0; i < N; i++) {
        printf("output[%d] = %.2f\n", i, h_output[i]);
    }

    cudaFree(d_input);
    cudaFree(d_mask);
    cudaFree(d_output);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 8$ nvcc q1_7.cu -o q1_7
pcap1@PC-36:~/Documents/240968040/week 8$ ./q1_7
Name: Devarjya, Reg No: 240968040
Launching CUDA kernel...

Result of 1D Convolution:
output[0] = 1.00
output[1] = 2.00
output[2] = 3.00
output[3] = 4.00
output[4] = 5.00
output[5] = 6.00
output[6] = 7.00
output[7] = 8.00
output[8] = 9.00
output[9] = 7.25
```

Q 1) Write a CUDA program that takes a string S as input and one integer value N. Produces output string N times as follows in parallel:
I/p: S = Hello N = 3
O/p String: HelloHelloHello (Each thread copies entire S)

Code:

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void repeat_string_kernel(const char *d_S, char *d_out, int N, int len) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < N) {
        int start_idx = tid * len;
```

```
        for (int j = 0; j < len; j++) {
            d_out[start_idx + j] = d_S[j];
        }
    }
}

int main() {
    printf("Name: Devarjya, Reg No 240968040\n");
    char h_S[] = "Hello";
    int N = 3;
    int len = strlen(h_S);

    int total_len = N * len;

    char *h_out = (char*)malloc((total_len + 1) * sizeof(char));

    char *d_S, *d_out;

    cudaMalloc((void**)&d_S, (len + 1) * sizeof(char));
    cudaMalloc((void**)&d_out, (total_len + 1) * sizeof(char));

    cudaMemcpy(d_S, h_S, (len + 1) * sizeof(char), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    printf("Launching kernel with %d threads...\n", N);
    repeat_string_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_S, d_out, N, len);


    cudaMemcpy(h_out, d_out, total_len * sizeof(char), cudaMemcpyDeviceToHost);

    h_out[total_len] = '\0';

    printf("I/p: S = %s, N = %d\n", h_S, N);
    printf("O/p String: %s\n", h_out);

    free(h_out);
    cudaFree(d_S);
    cudaFree(d_out);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 8$ nvcc q1.cu -o q1
pcap1@PC-36:~/Documents/240968040/week 8$ ./q1
Name: Devarjya, Reg No 240968040
Launching kernel with 3 threads...
I/p: S = Hello, N = 3
O/p String: HelloHelloHello
```

Q2) Write a CUDA program which reads a string consisting of N words and reverse each word of it in parallel

Code:

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void reverse_words_kernel(char *d_str, int len) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < len) {
        if (i == 0 || d_str[i - 1] == ' ') {
            int start = i;
            int end = start;
            while (end < len && d_str[end] != ' ' && d_str[end] != '\0') {
                end++;
            }
            end--;

            while (start < end) {
                char temp = d_str[start];
                d_str[start] = d_str[end];
                d_str[end] = temp;
                start++;
                end--;
            }
        }
    }
}

int main() {
   printf("Name: Devarjya, Reg No: 240968040\n");
    char h_str[] = "Hello World from CUDA programming";
    int len = strlen(h_str);

    char *d_str;

    cudaMalloc((void**)&d_str, (len + 1) * sizeof(char));

    cudaMemcpy(d_str, h_str, (len + 1) * sizeof(char), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (len + threadsPerBlock - 1) / threadsPerBlock;
```

```
    printf("Original String: %s\n", h_str);
    reverse_words_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_str, len);

    cudaMemcpy(h_str, d_str, (len + 1) * sizeof(char), cudaMemcpyDeviceToHost);

    printf("Reversed Words:  %s\n", h_str);

    cudaFree(d_str);

    return 0;
}
```

Output

```
pcap1@PC-36:~/Documents/240968040/week 8$ nvcc q2.cu -o q2
pcap1@PC-36:~/Documents/240968040/week 8$ ./q2
Name: Devarjya, Reg No: 240968040
Original String: Hello World from CUDA programming
Reversed Words:  olleH dlroW morf ADUC gnimmargorp
```

Q3) Write a program in CUDA to count the number of times a given word is repeated in a sentence. (Use Atomic function)

Code:

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void count_word_kernel(const char *sentence, const char *word, int lenS, int lenW, int *d_count) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i <= lenS - lenW) {
        int match = 1;

        for (int j = 0; j < lenW; j++) {
            if (sentence[i + j] != word[j]) {
                match = 0;
                break;
            }
        }

        if (match) {
            atomicAdd(d_count, 1);
        }
    }
}

int main() {
    printf("Name: Devarjya, Reg No: 240968040\n");
```

```
    char h_sentence[] = "CUDA is powerful and CUDA is fast because CUDA is parallel.";
    char h_word[] = "CUDA";

    int lenS = strlen(h_sentence);
    int lenW = strlen(h_word);

    int h_count = 0;

    char *d_sentence, *d_word;
    int *d_count;

    cudaMalloc((void**)&d_sentence, (lenS + 1) * sizeof(char));
    cudaMalloc((void**)&d_word, (lenW + 1) * sizeof(char));
    cudaMalloc((void**)&d_count, sizeof(int));

    cudaMemcpy(d_sentence, h_sentence, (lenS + 1) * sizeof(char), cudaMemcpyHostToDevice);
    cudaMemcpy(d_word, h_word, (lenW + 1) * sizeof(char), cudaMemcpyHostToDevice);
    cudaMemcpy(d_count, &h_count, sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (lenS + threadsPerBlock - 1) / threadsPerBlock;

    printf("Searching for word \"%s\" in sentence...\n", h_word);
    count_word_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_sentence, d_word, lenS, lenW,
d_count);

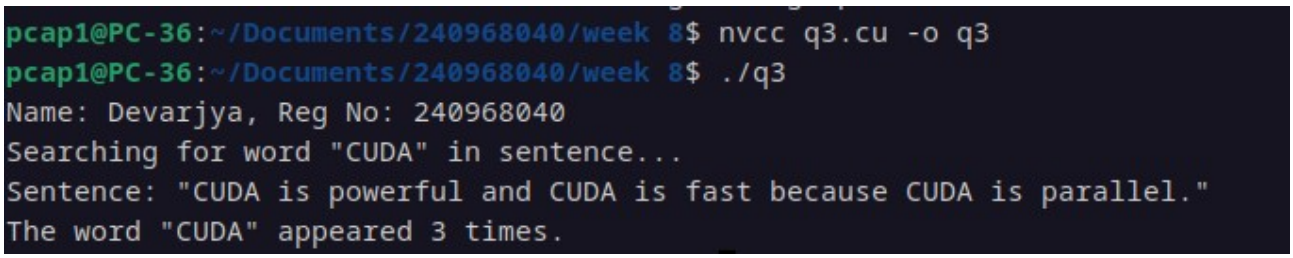
    cudaMemcpy(&h_count, d_count, sizeof(int), cudaMemcpyDeviceToHost);

    printf("Sentence: \"%s\"\n", h_sentence);
    printf("The word \"%s\" appeared %d times.\n", h_word, h_count);

    cudaFree(d_sentence);
    cudaFree(d_word);
    cudaFree(d_count);

    return 0;
}
```

Output:



Q4) Write a CUDA program that reads a string with N words and reverse entire string in parallel

Code:

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>

__global__ void reverse_entire_string_kernel(char *d_str, int len) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < len / 2) {
        int swap_idx = len - 1 - i;

        char temp = d_str[i];
        d_str[i] = d_str[swap_idx];
        d_str[swap_idx] = temp;
    }
}
int main() {
    printf("Name: Devarjya, Reg No: 240968040\n");
    char h_str[] = "Hello World from CUDA programming";
    int len = strlen(h_str);

    char *d_str;

    cudaMalloc((void**)&d_str, (len + 1) * sizeof(char));

    cudaMemcpy(d_str, h_str, (len + 1) * sizeof(char), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int num_threads = len / 2;
    int blocksPerGrid = (num_threads + threadsPerBlock - 1) / threadsPerBlock;

    printf("Original String: %s\n", h_str);
    reverse_entire_string_kernel<<<blocksPerGrid, threadsPerBlock>>>(d_str, len);

    cudaMemcpy(h_str, d_str, (len + 1) * sizeof(char), cudaMemcpyDeviceToHost);

    printf("Reversed String: %s\n", h_str);

    cudaFree(d_str);
    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 8$ nvcc q4.cu -o q4
pcap1@PC-36:~/Documents/240968040/week 8$ ./q4
Name: Devarjya, Reg No: 240968040
Original String: Hello World from CUDA programming
Reversed String: gnimmargorp ADUC morf dlroW olleH
```